Module 2 — Laboratory: User Stories and Use Cases for Your Project

Vistula University · IT Fundamentals · Laboratory assignment

Estimated time: 45–90 minutes **Mode:** Work in pairs (the same partner as in Module 1)

Prerequisites

- Module 1 lab is completed: you already have a GitHub repository with a project page (README).
- You have Git and (optional) VS Code installed, or you can edit files directly on github.com.
- You have read the Module 2 theory handout (Agile, Scrum, user stories, use case diagrams).

Goal of the lab

In Module 1, you made a project page. Now you will think like an Agile team: you will describe *who* uses your product, *what* they want to do, and *how* you will check that it works. This is exactly what a Scrum team does before starting a Sprint.

Tasks

Task 1 — Create a feature branch (5 min)

A "branch" is a separate line of work, so the main page stays safe while you edit.

1. Open your repository on GitHub (or your terminal).
2. Create a new branch called `module-2-requirements`.
3. Switch to it before making any changes.

Task 2 — Add a project description (5 min)

At the top of your `README.md`, add (or improve) a short description of your project:

- What the product is, in 2–3 simple sentences.
- Who will use it.
- What problem it solves.

Task 3 — Write at least 3 user stories (15 min)

Add a section called `## User stories`. Write at least 3 user stories in the exact format from the theory:

> As a *(kind of user)*, I want *(some goal)*, so that *(benefit)*.

Example for a study-planner app:

> As a student, I want to see all my tasks for today on one screen, so that I know what to do next.

Under each story, add at least 2 **acceptance criteria** as a short checklist, for example:

- [ ] The user can open the "Today" page with one click.
- [ ] Tasks are sorted by time.

Task 4 — List actors (5 min)

Add a section `## Actors`. Name at least **2 actors** for your project. Remember: an actor is a person or a system *outside* your system that uses it. Write one sentence for each actor saying what they do with your product.

Task 5 — List use cases (10 min)

Add a section `## Use cases`. Write at least **5 use cases**. Each use case is one thing an actor can do, named with a verb, for example: "Borrow a book", "Reset password". For each use case, note which actor(s) use it.

Task 6 — Draw a simple use case diagram (15–25 min)

Add a section `## Use case diagram` and include **one simple UML use case diagram** in your README. Choose any of these options:

- **Mermaid** (easiest — renders directly on GitHub). Mermaid cannot draw true ovals and stick persons, so use a flowchart-style diagram like the one in the theory handout, and write above it: *"Simplified use case diagram — actor nodes = stick persons, oval nodes = use cases."*
- **PlantUML** (has real stick-person and oval notation; render the image and add it with `![Use case diagram](images/use-case.png)`).
- **Any simple drawing tool** (draw.io, PowerPoint, even a clean photo of a hand drawing). Export as PNG, put the file in an `images/` folder, and embed it in the README.

Your diagram must show: the actors, at least 5 use cases, a system boundary, and lines from actors to their use cases.

Task 7 — List planned features (5 min)

Add a section `## Planned features (Product Backlog)`. Write 6–10 features as a simple ordered list. Put the most important feature first — you are ordering your Product Backlog like a Product Owner.

Task 8 — Commit, pull request, review and merge (15 min)

Now use the workflow you learned in Module 1:

1. Commit your changes with a clear message, for example: `Add user stories, actors, use cases and diagram`.
2. Push the branch and open a **pull request** (PR) to `main`.
3. Your **partner reviews** the PR: they read the user stories and check the format, the number of use cases, and whether the diagram is readable. They leave at least one comment (a question or a suggestion).
4. If the reviewer asks for a change, fix it and push again.
5. The reviewer **merges** the pull request. Then swap roles if your partner did their work on a separate branch.

Expected Deliverables

At the end of the lab, the repository must contain:

1. An updated `README.md` with: a project description, at least 3 user stories (correct format, each with at least 2 acceptance criteria), at least 2 actors with descriptions, at least 5 use cases with their actors, one use case diagram image (embedded), and an ordered list of planned features.
2. A merge commit showing that the work went through a feature branch and a reviewed pull request.
3. Review comments from the partner visible in the pull request.

Submission Instructions

1. Push all your work to GitHub and make sure the pull request is merged into `main`.
2. Check the README renders correctly on github.com (the diagram image is visible, lists look correct).
3. Submit through the course platform on the assignment page (Assignments section): paste your GitHub repository URL / deployed app URL / PR URL as required. - The **URL of your repository**. - The **URL of the merged pull request** (optional but welcome). - **Both partners' names and student ID numbers.**

Deadline: see the course platform. Late submissions may lose points.